On-Demand Schema Resolution for Tool-Calling LLM Agents

Omar El Ansary
Master Thesis Proposal

June 5, 2026

Abstract

Tool-calling LLM agents need schema information to produce valid function calls, but full-schema prompting scales poorly as tool pools grow. It increases input-token cost, makes tool selection harder in the presence of similar APIs, and can raise latency when schemas are repeatedly retrieved or expanded.

This thesis studies an *active tool registry*: a session-local memory of permitted and likely useful tools. Each entry stores a compact, schema-derived interface plus a pointer to the full schema. A router validates drafted calls and lets the agent execute from compact interfaces when validation passes, expand the full schema when more detail is needed, or abstain when no suitable tool is available. The evaluation compares this architecture with full-schema prompting and strong structural baselines using tool-call correctness, invalid-call rate, token usage, expansion rate, abstention quality, and average latency.

1 Introduction

An LLM agent typically needs precise information about a tool's name, arguments, types, and schema constraints to call it reliably. In small examples, this is usually handled by listing all available tools in the system prompt. However, as agentic environments scale to large tool pools [14], this static approach creates a three-way scaling bottleneck: cost, accuracy, and latency.

First, agentic workflows are input-heavy. In multi-turn settings, the model repeatedly processes system instructions, dialogue history, tool descriptions, previous tool calls, and tool outputs. Recent analyses of agentic coding tasks report that input tokens can drive overall token consumption and cost [1]; production cost analysis of long agentic coding sessions estimates that input tokens can outnumber output tokens by roughly 20:1 to 25:1 as context accumulates across turns [6]. Even when input tokens are cheaper than output tokens, their volume can dominate total inference cost. Reducing the input-token footprint of tool schemas is therefore a direct lever for lowering system cost.

Second, large tool contexts degrade tool selection and argument generation. Long-context function-calling evaluations report performance drops ranging from 7% to 85% as the number of available tools increases [7]. When semantically similar tools are shown together, the model must distinguish between APIs that may have similar descriptions but different parameter names, types, and constraints. This increases the risk of selecting the wrong tool, omitting required arguments, or hallucinating invalid parameters. Tool-use evaluation therefore requires not only choosing the right API, but also producing arguments that satisfy the target schema [12].

A common alternative is to retrieve tool schemas only when they seem relevant. Retrieval reduces the initial prompt size, but it can repeatedly fetch the same tools across turns and may add retrieval or prompt-processing latency. If retrieval returns only schema fragments, it may further reduce context cost but omit structural details needed for a valid function call. To address these bottlenecks, this thesis studies an *active tool registry*: a dynamic, session-scoped memory that exposes only permitted, compact tool interfaces and expands the full schema only when validation or optional uncertainty signals indicate that more detail is needed.

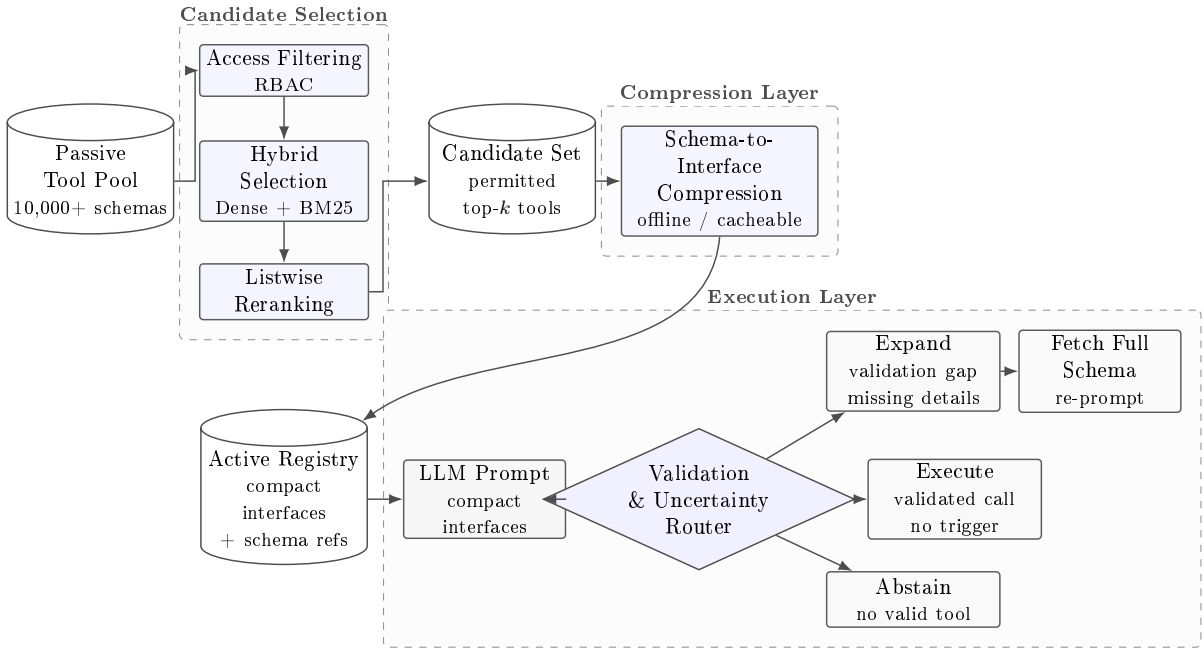


Figure 1: On-demand schema resolution pipeline for active registry construction and uncertainty-aware execution.

2 Problem Statement and Research Questions

This thesis studies the registry layer between retrieval and prompting for tool-calling LLM agents. Given a user request, access metadata, and a large *passive tool pool*, understood as the external catalogue of full tool schemas, the system must select permitted candidate tools and compile them into an *active registry*: a session-scoped set of compact interfaces with pointers to the corresponding full schemas. The central challenge is to retain enough schema structure in the prompt for valid tool calls while reducing the token and latency costs of repeatedly exposing full schemas. The thesis does not study autonomous tool synthesis or a complete access-control framework; access filtering is treated as a preprocessing step so that selection only operates over tools the agent is allowed to use or expose.

The contribution is therefore not a new access-control system or a new general-purpose retriever. Retrieval and access filtering are supporting components. The research contribution is the registry layer itself: how selected tools are represented compactly across a session, when that representation is sufficient for execution, and when the system should expand or abstain.

The main research question is:

> *Can a session-scoped active registry of compact tool interfaces reduce context usage and average latency while preserving tool-call correctness compared with full-schema prompting and strong structural baselines?*

This leads to three sub-questions:

- How should the system choose which tools enter the active registry?
- How much of a JSON/OpenAPI schema can be compressed into a compact interface without losing the information needed for a valid tool call?
- When should the system execute directly, load the full schema, or stop and report that no suitable tool is available?

3 Related Work and Research Gap

Existing work has made tool use by LLMs measurable from several angles. API-Bank evaluates executable multi-step tool use with 73 API tools, 314 annotated dialogues, and 753 API calls [9]. ToolLLM and ToolBench scale this setting to 16,464 REST APIs across 49 categories, making them useful for studying large-catalogue selection pressure [14]. BFCL focuses on function-calling correctness by checking whether a generated call matches the expected function and argument structure [12]. LongFuncEval evaluates the additional pressure created by long tool contexts, tool responses, and growing conversations [7]. ToolSandbox adds stateful, interactive tool-use scenarios with insufficient-information cases, making it relevant for abstention evaluation [10]. These benchmarks are useful because they separate tool selection, structural validity, long-context effects, and cases where the agent should not call a tool.

Other work studies how agents retrieve or manage tools during execution. Re-Invoke addresses zero-shot tool retrieval, SkillRouter studies retrieve-and-rerank selection over large skill collections, and MemTool studies short-term memory management for dynamic tool calling in multi-turn conversations [2, 19, 11]. Work on uncertainty-aware agents and tool-use abstention motivates the idea that an agent should sometimes defer instead of acting immediately [5, 16].

However, these lines of work leave open a narrower design problem: the registry layer between retrieval and prompting. This includes which tools should remain in session memory, how much schema information should be shown, when the full schema should be expanded, and when the system should abstain because the available tools are insufficient. This problem also requires explicit schema boundaries, since real JSON Schema features are not equally easy to compress or enforce during structured generation [3].

4 Proposed Architecture

Figure 1 summarizes the pipeline. The following subsections specify how candidate tools are selected, how selected schemas become active-registry interfaces, and how the router decides between execution, expansion, and abstention. The selection layer remains a strong supporting component; the main object of study is how selected tools are represented, maintained, and used after candidate discovery.

4.1 Hybrid Selection and Reranking

Before schema compression, the system must decide which tools should be candidates for the active registry. Recent tool-selection literature suggests that large-scale tool routing benefits from multi-stage retrieval rather than a single dense search step [2, 19]. The goal is not to identify the final tool call directly, but to construct a small candidate set that is likely to contain the tools needed for the current task.

Access filtering. The passive tool pool is first reduced by access metadata such as allowed agent roles, user permissions, and deployment environment. Tools that fail this check are removed before relevance ranking. This keeps authorization separate from retrieval: a retriever should not be responsible for hiding tools that the agent is not allowed to use.

Hybrid candidate retrieval. The remaining tools are ranked using both lexical and semantic signals. Sparse retrieval, such as BM25, can capture exact tool names, parameter names, and API-specific terms, while dense retrieval can capture broader task similarity [15, 2]. To avoid relying only on surface-level descriptions, the retrieval index can include structured schema information such as tool names, descriptions, parameter names, argument types, and selected constraints. This richer retrieval representation is used for selection; it is not inserted into the model prompt by default.

Reranking and candidate formation. The initial candidate set can then be reranked using schema-aware scores or a compact reranker. SkillRouter is one example of a retrieve-and-rerank

approach showing that richer tool or skill bodies can improve selection in large, overlapping tool collections [19]. The top-ranked tools become the candidate set passed to compression. Candidate-set size remains an experimental variable: a small set may miss useful tools, while a large set may reintroduce the context problem.

4.2 Schema-to-Interface Compression and Boundaries

Once tools are selected, their full schemas should not automatically be placed in the prompt. The thesis will compile each selected JSON/OpenAPI schema into an active-registry entry: a compact interface that keeps the information most relevant for function calling and a pointer to the full schema outside the prompt. To make this representation measurable, compression will use four fixed levels rather than ad hoc summaries:

- **L0 (minimal):** tool name and short description only; used for candidate selection and ablations, but not executable.
- **L1 (flat structural):** L0 plus required and optional primitive arguments with types; executable only for flat primitive tools.
- **L2 (descriptive):** L1 plus enum values, default values, shallow primitive arrays, and short parameter descriptions.
- **L3 (shallow constrained):** L2 plus one-level nested objects, nullability, and simple constraints; the last compressed level before full-schema expansion.

Retrieval and compression are treated as separate steps. Retrieval may index rich schema information to improve candidate selection, but the active prompt receives only the compact interface. The compiler preserves schema boundaries instead of chunking JSON text arbitrarily, so the levels above can be compared systematically.

The primary evaluation will also define explicit schema boundaries. Compact-safe features include tool names, descriptions, primitive types, required/optional flags, enum values, default values, and shallow primitive arrays. Features that can affect validity but are too detailed for compact execution, such as nested objects beyond one level, complex arrays, detailed examples, regex patterns, and strict length or range constraints, trigger expansion to the full schema. Recursive references, cyclic schemas, external `$ref` chains, complex provider-specific extensions, and polymorphic operators such as `oneOf`, `anyOf`, and `allOf` will be normalized or excluded from the primary evaluation rather than silently compressed.

Where the model-serving software, for example vLLM [8] or a comparable serving API, supports prompt-prefix caching, the active registry can be kept as a stable prompt prefix. Full schemas requested during expansion can then be appended after this prefix and removed after use. This can preserve reuse of the stable prefix, but expanded schemas may still add prefill and validation overhead. If the model server reports cache usage or separates prompt-processing time from generation time, the evaluation will report cached and uncached results separately; otherwise, latency will be reported end-to-end.

4.3 Routing Under Uncertainty

The router decides whether the active registry is sufficient. Before generation, it checks whether the ranked candidate set contains any plausible tool. If no candidate passes a minimum relevance threshold, the system reports that no available tool seems suitable. Otherwise, the model sees the active registry and drafts a tool call.

After the draft, the router first applies offline syntactic and schema validation to the generated JSON arguments, for example with JSON Schema or Pydantic-style checks. Validation does not mean executing the tool or probing a live API. Confidence signals are treated as optional and experimental: where the model server or API exposes token log-probabilities, the system may inspect the generated function name and argument keys; where log-probabilities are unavailable,

multi-sample agreement may be measured as a costlier fallback. This caution is supported by semantic-parsing calibration work and by recent structured-JSON confidence results, where 99.4–100% of token log-probabilities saturated above 0.999 in some structured outputs, leaving little signal for a simple threshold rule [17, 13]. The router then chooses one of three actions:

- **Execute:** call the selected tool if the draft call passes validation and no expansion trigger is active.
- **Expand:** load the full schema if the selected tool is relevant but validation fails in a way that may be caused by omitted schema detail, or if optional uncertainty signals indicate that compact information is insufficient.
- **Abstain:** report that no available tool seems suitable if no candidate passes the relevance threshold, or if the drafted call remains unsupported after validation and expansion checks.

This policy keeps the execute, expand, and abstain decisions measurable. The validation-only router is the default baseline because it is deterministic and available across serving setups. Log-probability thresholds, top-candidate margins, or repeated-sampling consistency will be evaluated only as auxiliary router signals and reported with their latency cost. Thresholds for relevance, execution, and expansion will be tuned on a development split and evaluated on held-out tasks. Prior work on uncertainty-aware agents motivates using confidence signals to decide when an agent should not execute immediately, but this thesis tests whether these signals help specifically with schema expansion and abstention in tool calling [5, 16].

5 Evaluation Plan

The primary success criterion is whether the proposed active registry improves the correctness-token Pareto trade-off against strong baselines while keeping tool-call correctness close to full-schema prompting where full schemas fit in context. Latency will be reported as a practical system trade-off rather than as a guaranteed consequence of token reduction.

The direct experimental comparison is between the proposed active registry and strong structural baselines rather than naive retrieval alone. The proposed condition uses the same selected candidate tools as the retrieval baselines, but compiles them into L0–L3 active-registry interfaces and routes each attempted call through execute, expand, or abstain. This isolates the effect of the registry layer from the effect of candidate retrieval. The baselines are:

1. **Full-schema prompting:** expose full schemas directly where prompt length permits.
2. **Stateful top-k full-schema retrieval:** retrieve full schemas and retain the same selected tools across session turns.
3. **Stateless top-k full-schema retrieval:** retrieve full schemas independently at each turn without session memory.
4. **Compact-only prompting:** force execution using compact interfaces without full-schema expansion.
5. **Validation-only router:** use syntactic and schema checks without logprob thresholds.

To isolate failure causes, errors will be decomposed into *selection failure* when the required tool is absent from the active registry, *compression failure* when the tool is present but the compact interface omits necessary detail, *router failure* when the execute/expand/abstain decision is wrong, and *validation failure* when the generated call violates the target schema. An oracle-registry ablation will guarantee that the gold tool is present, so compression and routing can be evaluated independently from retrieval.

The primary benchmark setup will use BFCL for structured function-call correctness [12] and API-Bank for executable multi-step dialogues [9]. ToolBench will be used for large-catalogue stress tests if a reproducible local setup is available [14, 4]. If ToolBench is not stable enough

for the thesis environment, a controlled synthetic stress test will vary the pool size, for example $N \in \{50, 100, 300, 1000\}$, and distractor similarity around BFCL/API-Bank-style tools. Negative cases will include no suitable tool, removed-gold-tool splits, semantically similar distractors, and missing required user information, so abstention can be measured with false execution and false abstention rates. The exact benchmark releases and generated pools will be fixed before the experiments to keep the results reproducible.

The experiments will use open-weight Qwen-family models [18]. The planned target model is Qwen3-32B served with vLLM on H100 or A100 GPU nodes, depending on available compute. Smaller Qwen models will be used for ablations where lower compute cost is needed. vLLM will be used to record latency and token log-probabilities when exposed by the serving configuration.

The main metrics are:

- **Registry coverage:** whether the required tool or tools are included in the active registry before expansion.
- **Tool-call correctness:** whether the agent selects the expected function and provides arguments that satisfy the target schema.
- **Invalid-call rate:** syntactically invalid calls and schema-validation failures.
- **Expansion rate:** how often the full schema is requested after seeing the active registry.
- **Abstention quality:** whether the system detects missing capabilities without false executions or unnecessary refusals.
- **Efficiency:** prompt tokens, retrieval and expansion calls, and end-to-end latency.

Latency will be interpreted as an average-case trade-off, not as a guaranteed reduction on every turn. The execute path uses one generation over a smaller prompt, while the expand path may require a second generation after loading the full schema. The thesis will therefore report latency separately for execute, expand, and abstain decisions, and will test whether savings on frequent execute decisions offset the extra cost of expansion. The ablation study will vary registry size, compression level, schema feature boundary, retained schema fields, and routing thresholds. These ablations will indicate where compression starts to hurt correctness, and when schema expansion is worth its additional cost.

6 Timeline (22 Weeks)

- **Weeks 1–4:** Build the smallest complete BFCL/BFCL-like pipeline: retrieval, compact interface, model call, validation, and optional full-schema expansion. Reproduce full-schema and top-k retrieval baselines on this subset.
- **Weeks 5–8:** Implement access filtering, hybrid retrieval, reranking, the schema-to-interface compiler, and L0–L3 compression levels. Run initial selection and compression ablations.
- **Weeks 9–12:** Implement the session-scoped active registry, validation-only router, and optional uncertainty-signal ablations. Tune routing thresholds on a development split.
- **Weeks 13–17:** Run the primary benchmark evaluations and, if setup is stable, large-catalogue stress tests. Measure tool-call correctness, invalid calls, expansion rate, abstention, token usage, and latency.
- **Weeks 18–22:** Analyze trade-offs, validate the registry interfaces in integration tests, and write the final thesis.

References

[1] Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How do ai agents spend your money? analyzing and predicting token consumption in agentic coding tasks. *arXiv preprint arXiv:2604.22750*, 2026.

[2] Yanfei Chen, Jinsung Yoon, Devendra Singh Sachan, Qingze Wang, Vincent Cohen-Addad, MohammadHossein Bateni, Chen-Yu Lee, and Tomas Pfister. Re-invoke: Tool invocation rewriting for zero-shot tool retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

[3] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Jsonschemabench: A rigorous benchmark of constrained decoding for structured generation. *arXiv preprint arXiv:2501.10868*, 2025.

[4] Zhicheng Guo et al. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[5] Jiuzhou Han, Wray Buntine, and Ehsan Shareghi. Towards uncertainty-aware language agent. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 6662–6685. Association for Computational Linguistics, 2024.

[6] Casey Harding. The hidden cost driver in agentic coding: It's not the per-token price. Vantage Blog, April 2026. Accessed May 30, 2026. Available at `https://www.vantage.sh/blog/agentic-coding-costs`.

[7] Kiran Kate, Tejaswini Pedapati, Kinjal Basu, Yara Rizk, Vijil Chenthamarakshan, Subhajit Chaudhury, Mayank Agarwal, and Ibrahim Abdelaziz. Longfunceval: Measuring the effectiveness of long context models for function calling. *arXiv preprint arXiv:2505.10570*, 2025.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[9] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. *arXiv preprint arXiv:2304.08244*, 2023.

[10] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025.

[11] Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah, Pradeep Honaganahalli Basavaraju, and James A. Burke. Memtool: Optimizing short-term memory management for dynamic tool calling in llm agent multi-turn conversations. *arXiv preprint arXiv:2507.21428*, 2025.

[12] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*, 2025.

[13] Jasmine Qi, Danylo Dantsev, and Muyang Sun. Verdi: Single-call confidence estimation for verification-based llm judges via decomposed inference. *arXiv preprint arXiv:2605.11334*, 2026.

[14] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

[15] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[16] Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. When2call: When (not) to call tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3391–3409, 2025.

[17] Elias Stengel-Eskin and Benjamin Van Durme. Calibrated interpretation: Confidence estimation in semantic parsing. *Transactions of the Association for Computational Linguistics*, 11:1213–1231, 2023.

[18] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[19] YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. Skillrouter: Skill routing for llm agents at scale. *arXiv preprint arXiv:2603.22455*, 2026.